

Security Audit

LFG Incorporated (Token)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	9
Intended Smart Contract Functions	10
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Status Definitions	16
Audit Findings	17
Centralisation	34
Conclusion	35
Our Methodology	36
Disclaimers	38
About Hashlock	39

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The LFG Incorporated team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

LFG Incorporated is a full-service project foundry and incubator that empowers visionary Web3 projects fund, accelerate, and scale. Its Core or Node Sale platform gives founders the ability to configure, manage, and launch fundraising with speed, flexibility, and complete transparency, featuring tiered pricing, automated price adjustments, and a referral system. By pairing hands-on mentorship with strategic resources and a powerful end-to-end launch solution, LFG enables projects to raise capital, build sustainable decentralized networks, and grow thriving communities.

Project Name: LFG Incorporated

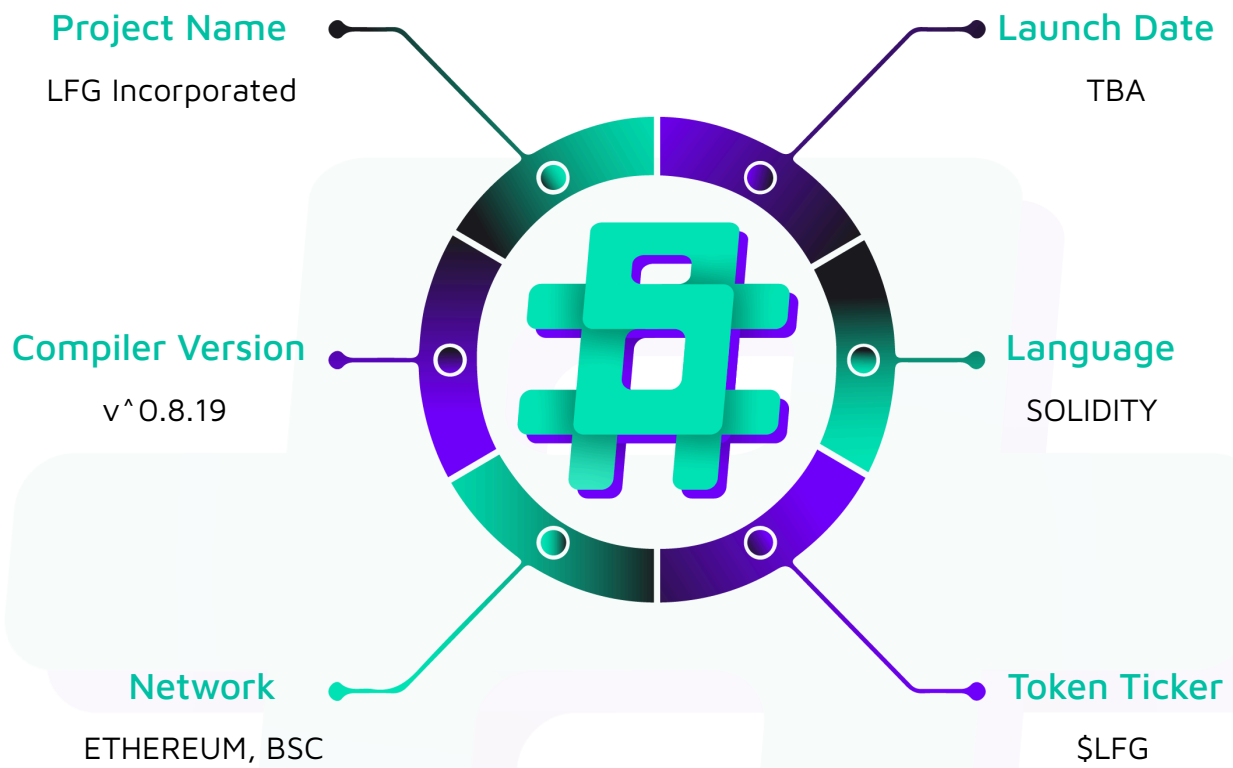
Project Type: Launchpad

Compiler Version: ^0.8.19

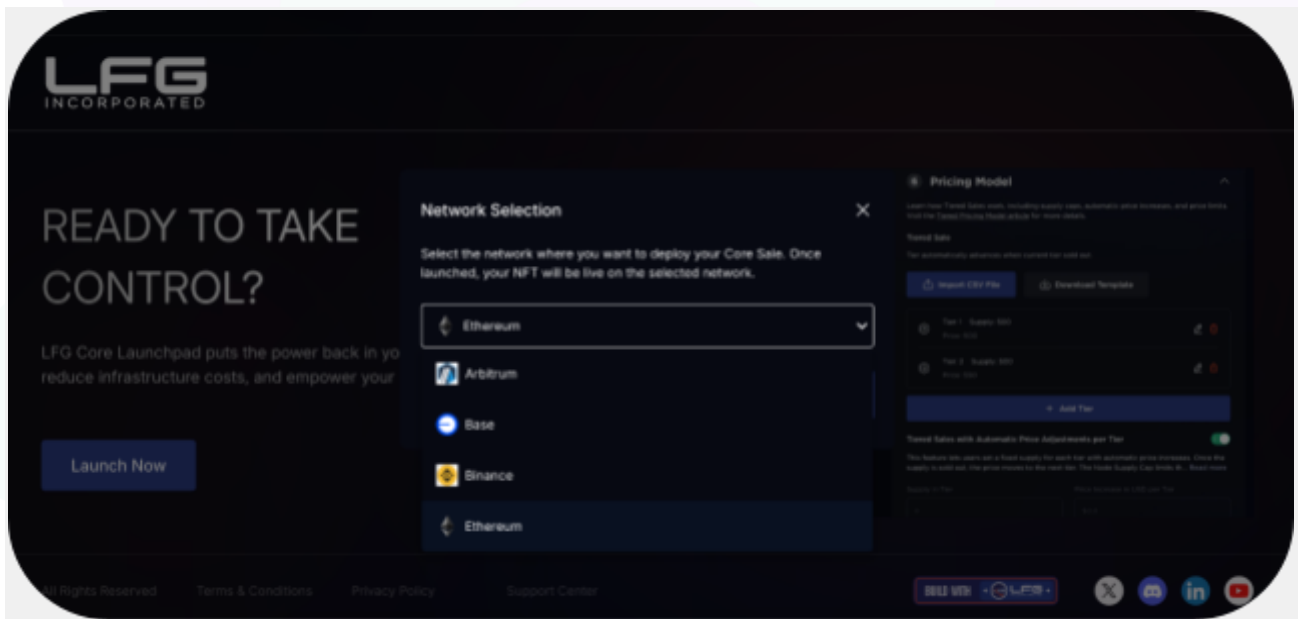
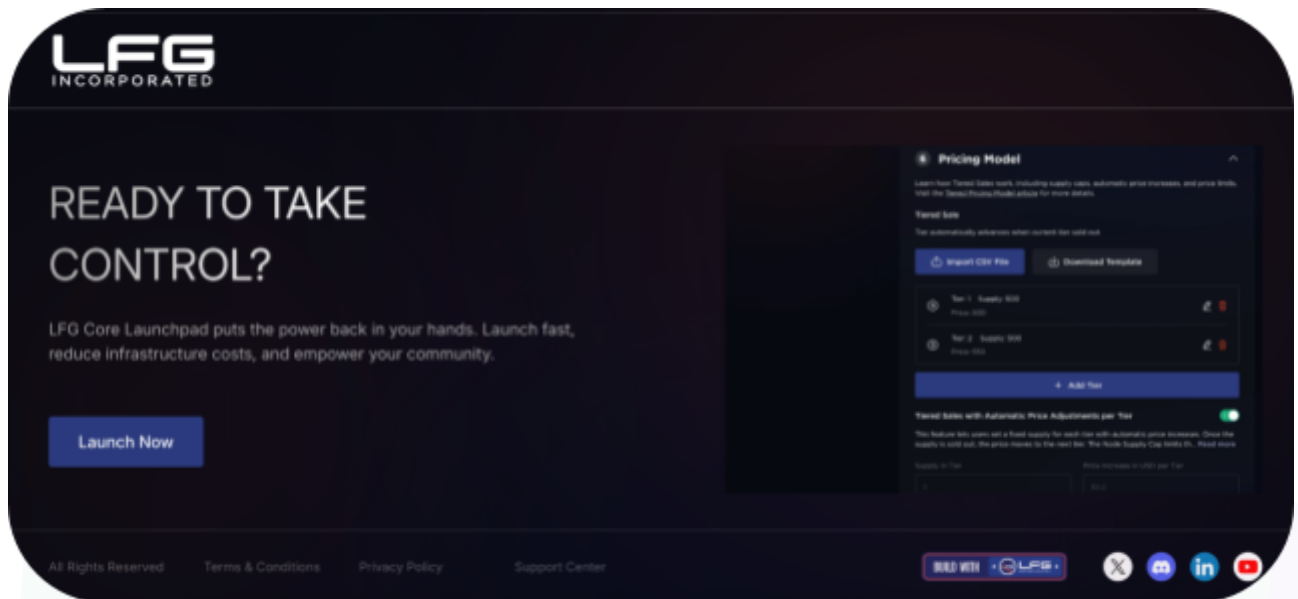
Website: <https://lfg.inc/>

Logo:



Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the LFG Incorporated project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	LFG Incorporated Smart Contracts
Platform	Ethereum / Solidity
Audit Date	June, 2025
Contract 1	Constants.sol
Contract 2	Factory.sol
Contract 3	Redeem.sol
Contract 4	Rewards.sol
Contract 5	Deployer.sol
Contract 6	IFactory.sol
Contract 7	INodeCallee.sol
Contract 8	IRewards.sol
Contract 9	ISaleCallee.sol
Contract 10	ITierPriceCallee.sol
Contract 11	Node1.sol
Contract 12	Sale1.sol
Contract 13	TierPrice1.sol
Contract 14	Payments.sol
Contract 15	Phases.sol
Contract 16	UserBuyInfo.sol

Audited GitHub Commit Hash	2428b674a7a2cca4e438105410a20c416f380f70
Fix Review GitHub Commit Hash	4129f701f4a6873281a3f63c67ca12ebf76ceae3



Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 High severity vulnerability

4 Medium severity vulnerabilities

4 Low severity vulnerabilities

3 Gas Optimisations

1 QA

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Factory.sol - Allows Admins to: - Grant and manage system-wide roles (e.g., CREATE_PROJECT_ROLE, PRICE_FEED_ROLE, REWARDS_SIGNER_ROLE, REDEEM_SIGNER_ROLE, DISTRIBUTE_FUND_ROLE). - Add and manage "deployer" contracts (implementations for Sale, TierPrice, Node contracts). - Enable or disable registered deployer contracts. - Set whether a DAO mechanism (useDAO) is active for project creation. - Configure LFG (Looking For Group/Guild) address and share percentage for fund distribution. - (Potentially, through specific role assignments) Change the Tier Price contract for an existing project. - Allows Designated Roles to: - Create new projects by deploying and initializing sets of Sale, TierPrice, and Node contracts using registered deployers.	Contract achieves this functionality.
Sale1.sol - Allow Users to:	Contract achieves this functionality.

- Buy NFTs by providing a maxUsdValue they are willing to pay and a valid, unexpired payment signature (PaymentPriceSignature) from an authorized PRICE_FEED_ROLE holder. Payment can be in ETH or accepted ERC20 tokens.
- Participate in Over-The-Counter (OTC) sales by providing a valid, unexpired OTC signature (OTCSignature) from an authorized OTC creator or SUPER_ADMIN_ROLE.
- Receive refunds for their purchases (if the sale fundState is set to REFUND and they haven't already refunded for a specific token).
- Allow Admins to:
- Initialize the sale (accepted payment tokens, start time, phase configurations, rewards percentage, redeem enabling).
- Set system addresses (Factory, TierPrice, NFT, Rewards, Redeem contracts, project ID).
- Enable or disable the redemption feature.
- Update the percentage of sales revenue allocated to rewards.
- Pause and unpause sale functionalities (like buy, otc, token management, phase management).
- Add new accepted payment tokens.
- Remove existing accepted payment tokens.
- Add new sale phases (defining party distributions and sale volume triggers).
- Update existing sale phases (modifying parties and their weights).
- Unlock funds for distribution (changing fundState from LOCK to UNLOCK). This also triggers funding the Rewards contract.

- Set the sale fundState to REFUND (allowing users to claim refunds). - Trigger distribution of accrued funds to parties for a specific phase (if fundState is UNLOCK). - Trigger distribution of accrued funds on behalf of a third party (with a fee, callable by DISTRIBUTE_FUND_ROLE). - Distribute royalty payments accrued from secondary market sales (if any mechanism outside this contract feeds them in). - Perform an emergency withdrawal of all funds (ETH or specific ERC20) from the Sale contract.	
TierPrice1.sol - Allow Admins to: - Define the initial pricing tiers: - Borders (cumulative sold amounts for each tier). - Prices for each tier. - Configure parameters for dynamic/automatic tier generation: - increaseAmount (how much the border increases for a new dynamic tier). - increasePrice (how much the price increases for a new dynamic tier). - Set an overall cap on the total number of items that can be sold/accounted for. - Set a priceCap for dynamically generated tiers. - Allow Sale1.sol to: - Request the total USD value for a given amount of NFTs to be sold, which also updates the internal sold count and	Contract achieves this functionality.

currentTier.

- "Cut" (account for as if sold, without actual payment) a specified amount of NFTs from the current pricing tier(s), updating sold and currentTier. This is used for OTC or redemptions that consume supply.
- "Cut" (reduce the availability of) a specified amount of NFTs from the highest tier by reducing the overall cap.

Code Quality

This audit scope involves the smart contracts of the LFG Incorporated project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the LFG Incorporated project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] Sale1&Phases - Changing Global rewardsPercent After Phase Creation Breaks Fund Distribution Logic

Description

The Phases.sol contract (used by Sale1.sol) establishes an invariant during phase creation (`_addPhase`) or update (`_updatePhase`): the sum of party weights (`totalWeight`) for a phase must equal `ONEHUNDRED_PERCENT - _rewardsPercent` at that moment (`_rewardsPercent` being the global `rewardsPercent` in `Sale1.sol`).

However, the `Sale1.sol` contract allows an administrator to change the global `rewardsPercent` at any time via the `updateRewardsPercent` function (which calls `_updateRewardsPercent`).

If `rewardsPercent` is changed *after* phases have already been configured with party weights based on a previous `rewardsPercent`, a discrepancy arises.

The fund distribution logic in `Sale1.sol`'s `distribute` and `distributeBehalf` functions calculate each party's share using the *current* global `rewardsPercent`:

```
tokenAmount = (partyWeight * amountToDistribute) / (ONEHUNDRED_PERCENT - rewardsPercent);
```

If the denominator (`ONEHUNDRED_PERCENT - rewardsPercent`) used here (based on the current global `rewardsPercent`) does not match the sum of `partyWeights` for that specific phase (which was set based on an older `rewardsPercent`), the distribution will be incorrect.

Vulnerability Details

Assume this scenario: Admin sets initial `rewardsPercent` = 10%.

1. Admin calls `addPhase` for Phase 1. Parties A and B are added with weights that sum up to 90% (i.e., `ONEHUNDRED_PERCENT - 10%`).
2. Admin later calls `updateRewardsPercent` to set global `rewardsPercent = 5%`.
3. Funds are collected, and distribution is called for Phase 1.
4. The distribution calculation will use $(\text{ONEHUNDRED_PERCENT} - 5\%) = 95\%$ as the denominator.
5. However, the actual sum of `partyWeights` for Phase 1 is 90%.
6. Since the denominator used (95%) is larger than the actual sum of weights (90%), the `tokenAmount` calculated for each party will be less than intended. The total amount distributed to parties in Phase 1 will be less than the `amountToDistribute` available for them, leading to funds being stuck.
7. Conversely, if `rewardsPercent` was increased (e.g., from 10% to 20%), the denominator would become smaller (80%), leading to over-distribution and potentially draining more funds than allocated to parties for that phase.

Impact

This vulnerability leads to incorrect calculation of fund distributions to parties within a phase. Depending on how `rewardsPercent` is changed relative to when phases were configured, parties may receive less than their entitled share (leading to stuck funds) or more than their entitled share (leading to a deficit or incorrect drawdown of the phase's party funds). This directly affects the financial integrity of the sale's fund distribution mechanism.

Recommendation

Disallow `rewardsPercent` Changes After Phases Exist:

```
// Sale1.sol
function _updateRewardsPercent(uint256 _newRewardsPercent) internal {
    require(totalPhase == 0, "Sale1: Cannot change rewards% after phases are added"); //
New check
    if (_newRewardsPercent > ONEHUNDRED_PERCENT)
        revert InvalidRewardsPercent();
    rewardsPercent = _newRewardsPercent;
    emit UpdateRewardsPercent(_newRewardsPercent);
```

```
}
```

Status

Resolved



Medium

[M-01] TierPrice1 - Can Enter a State Where cap Cannot Be Further Reduced

Description

The `TierPrice1` contract, which manages pricing tiers and total supply (`cap`) for a sale, can enter a state where the `cap` can no longer be administratively reduced. This occurs if the number of items sold (or administratively "cut" using `cutFromCurrentTier`) becomes equal to the current `cap`.

The sequence is as follows:

1. An admin first calls `cutFromHighestTier(amount_A)` to lower the `cap` to a new value, `cap_new`. This `cap_new` might be set below some of the originally defined tier borders, but must be $\geq$ `sold`.
2. Then, the admin calls `cutFromCurrentTier(amount_B)` where $\text{amount_B} = \text{cap_new} - \text{sold_current}$. This operation increments until `sold == cap_new`.

After this point, if the admin attempts to call `cutFromHighestTier(amount_C)` with $\text{amount_C} > 0$, the transaction will revert. This is because the check if $(\text{new_calculated_cap} < \text{sold} \dots)$ inside `cutFromHighestTier` will always be true (as `sold` equals the `cap` before this new attempted reduction).

3. Similarly, `cutFromCurrentTier` will have no effect as its primary loop condition $\text{sold} < \text{cap}$, will be false.

Once `sold == cap`, the `cap` value is effectively frozen from further administrative reductions, and no more items can be sold or "cut."

Impact

If an administrator uses the `cutFromCurrentTier` and `cutFromHighestTier` functions to fine-tune supply and inadvertently sets `sold` to be equal to `cap`, they lose the ability to further reduce the `cap`. While the sale is effectively concluded at this point (no more items can be sold), the inability to adjust the `cap` downwards post-factum (e.g., to signal a smaller final supply than previously indicated by the `cap`) can be a significant

operational limitation. This results in a loss of administrative flexibility for a key parameter. It does not lead to direct loss of funds but can hinder strategic supply management.

Recommendation

The current behavior of disallowing `cap < sold` is generally correct to maintain consistency. The issue arises from the "terminal" nature of `sold == cap` for cap adjustments.

Consider if there's a need for an admin to *ever* reduce cap once `sold == cap`.

If this scenario is deemed an undesirable edge case lock:

Documentation and Admin Guidance: Clearly document this behavior so administrators are aware that `cutFromCurrentTier` up to the current cap will finalize the cap against further reductions via `cutFromHighestTier`.

Allowing Reduction when `sold == cap` (with caution): If there's a strong need to reduce cap even when `sold == cap` (implying the "sold" count was also intended to be retrospectively capped), the logic in `cutFromHighestTier` could be slightly altered. This is risky as it might imply `cap < sold` if not handled carefully.

A potential, though complex, change might also involve reducing `sold` if the cap is reduced below the current `sold` value, but this has significant implications and would require careful thought on what "sold" means if it can be administratively reduced after the fact.

The simplest safe approach might be the check if `(cap < sold ...)` remains, and admins must be aware of its implications.

The audit comment regarding if `(cap == 0) revert CannotCutFromHighestTier();` in `cutFromHighestTier` (suggesting validation with `_amount`) refers to a different concern: if `cap == 0` signifies an "unlimited" cap due to `increaseAmount > 0` and no explicit `_cap` in `initialize`. In such an "unlimited" scenario, `cutFromHighestTier` might not be meaningful. However, the `initialize` function aims to set cap to a non-zero value:

- If `increaseAmount == 0` (fixed tiers), `cap = _border[_border.length - 1]`.

- If `increaseAmount != 0` (dynamic tiers) and an explicit `_cap > 0` is passed to `initialize`, then `cap` takes that value.

If `increaseAmount != 0` and `_cap == 0` is passed to `initialize`, then the state variable `cap` remains `0`, indicating truly dynamic, uncapped (by quantity) tiers. In this specific state (`state.cap == 0`), calling `cutFromHighestTier` might be problematic (e.g. `0 - _amount` underflows). The current `if (cap == 0) revert` correctly prevents this.

The note about `_cap <= _border[_border.length - 1] + _increaseAmount` in `initialize` where the auditor suggests strictly less than, is a matter of interpretation of how `InvalidCap` should be triggered based on the very first potential dynamic tier's end quantity versus the overall cap. The current code requires `_cap` to be strictly greater than the end of the first dynamic tier extension. This seems like a reasonable constraint to ensure an explicit cap is meaningful if dynamic increases are enabled.

No specific code change is immediately obvious for the main described "lock" issue without fundamentally changing what `cap` and `sold` represent or adding complexity. The primary recommendation is thorough documentation of this interaction for administrators.

Status

Resolved

[M-02] Factory#addSaleDeployer&addTierPriceDeployer&addNodeDeployer - Missing Contract Validation for Implementation Addresses

Description

The admin-controlled functions `addSaleDeployer`, `addTierPriceDeployer`, and `addNodeDeployer` in `Factory.sol` are used to register addresses intended as implementation contracts for proxies created via `Clones.cloneDeterministic()`. These functions do not verify that the provided `_deployer` address contains contract code and is not an Externally Owned Account (EOA).

Impact

If a `SUPER_ADMIN_ROLE` holder mistakenly registers an EOA as a deployer address, subsequent calls to `createProject` attempting to use this EOA for `Clones.cloneDeterministic()` will fail. This will cause project creation transactions to revert, disrupting a core functionality of the platform until the incorrect EOA deployer is identified and disabled or corrected.

Recommendation

Before adding a `_deployer` address to the respective arrays, validate that it is a contract by checking `_deployer.code.length > 0`. (Apply similar checks to `addTierPriceDeployer` and `addNodeDeployer`)

```
// Example for addSaleDeployer
function addSaleDeployer(
    address _deployer,
    string calldata _description // Assuming 'calldata' fix from Finding 11
) external onlyRole(SUPER_ADMIN_ROLE) {
    require(_deployer != address(0), "Factory: Deployer cannot be zero address"); // For Finding 5
    require(_deployer.code.length > 0, "Factory: Deployer must be a contract");
    if (deployerAdded[_deployer]) revert DeployerAlreadyExist();
    // ... rest of function logic
}
```

Status

Resolved

[M-03] Factory#Constructor - `SUPER_ADMIN_ROLE` is Directly Assigned to Supplied Parameter instead of `msg.sender`

Description

The Factory contract's constructor assigns the `SUPER_ADMIN_ROLE` directly to the `_admin` address provided as a parameter. It does not first assign this role to `msg.sender` (the contract deployer).

If the deployer provides an incorrect address (e.g., typo, wrong address, or `address(0)`) for the `_admin` parameter during deployment.

1. The `SUPER_ADMIN_ROLE` is granted to this incorrect/zero address.
2. The deployer (`msg.sender`) does not receive the `SUPER_ADMIN_ROLE`.

Impact

This can lead to an immediate and potentially irreversible loss of administrative control over the Factory contract if the `_admin` address is incorrectly specified during deployment. If assigned to `address(0)`, super admin functions become permanently inaccessible. If assigned to an unintended EOA, control is wrongly delegated. This critically compromises the entire system managed by the Factory.

Recommendation

It is a safer pattern to grant the `SUPER_ADMIN_ROLE` (and OpenZeppelin's `DEFAULT_ADMIN_ROLE`) to `msg.sender` (the deployer) first within the constructor. If a different `_admin` is intended, the deployer can then transfer the role in a subsequent transaction, or the constructor can grant it to the `_admin` parameter *additionally*, ensuring the deployer is not locked out.

Status

Acknowledged

[M-04] Factory#Constructor - Missing Zero-Address Validation Along with No Setter Functions leads to Potential Loss of Control

Description

The Factory contract's constructor accepts several address parameters (`_admin`, `_projectCreator`, `_priceFeeder`, `_rewardsSignerRole`, `_redeemSigner`, `_fundDistributor`) which are then granted critical roles. The constructor does not validate if any of these input addresses are `address(0)` before the role assignments.

Impact

Assigning critical roles to `address(0)` during deployment can lead to a permanent loss of administrative control over the Factory (if `SUPER_ADMIN_ROLE` is affected) or render key system functionalities (like price-signed purchases, reward claims, or redemptions) inoperable. This severely compromises the platform's manageability and intended operations. As there are no functions to update these values, it becomes more severe.

Recommendation

Implement strict zero-address validation for all address parameters in the constructor *before* granting any roles. Add setter functions to change these values too.

Status

Resolved

Low

[L-01] Payments#_addTokens&_removeTokens - Batch Token Operations Revert Automatically on Single Item Failure

Description

The internal functions `_addTokens` and `_removeTokens` are designed to process an array of token addresses. If any single token in the provided array fails its respective validation check (e.g., a token in `_addTokens` is already in the `_tokenAccepted` mapping, or a token in `_removeTokens` is not currently in `_tokenAccepted`), the entire transaction reverts. This means that any operations on tokens processed earlier in the loop within the same transaction are also rolled back, ensuring atomicity for the batch.

For example, if an admin attempts to call `_addTokens` with `[tokenA, tokenB, tokenC]` and `tokenB` is already an accepted token, the function will revert with `TokenAlreadyAdded(tokenB)`. As a result, `tokenA` (which might have been valid and processed before `tokenB` in the loop) will also not be added.

Recommendation

Simply Iterate and Skip failed items.

```
// event TokenAdditionFailure(address indexed token, string reason); // Define this event
// Inside _addTokens loop
// if (_tokenAccepted[_tokens[i]]) {
//     emit TokenAdditionFailure(_tokens[i], "TokenAlreadyAdded");
//     continue; // Skip to the next token
// }
// ... rest of the logic for adding a valid token
```

Status

Resolved

[L-02] Node1#mint&safeMint - Mint & safeMint functions does not strictly follow the Checks-Effects-interactions pattern

Description

The `safeMint` function in `Node1.sol` mints multiple tokens in a loop. It reads `_idCounter` at the start of the loop (`uint256 tokenId = _idCounter;`), calls `_safeMint` (which has its own effects and a potential external interaction `_checkOnERC721Received`), increments the local `tokenId`, and only updates the state variable `_idCounter` *after* the loop completes.

If `_checkOnERC721Received` (called by `_safeMint`) were to re-enter `Node1` and read `_idCounter`, it would see the value of `_idCounter` before the current batch operation started incrementing it for all items in the batch. For ERC721 `_safeMint` with its internal `_exists` checks, a direct re-minting exploit of the same ID within the batch is unlikely. However, not updating `_idCounter` (the Effect) fully before all Interactions in the loop is a deviation from strict CEI for batch operations. It's primarily a code clarity and best practice issue here rather than a direct exploit, given ERC721 mechanics.

Recommendation

To adhere more strictly to the CEI pattern for batch state updates, update the state variable `_idCounter` to its final value for the batch *before* the loop containing the interactions.

```
// Node1.sol
function safeMint(
    address to,
    uint256 amount,
    bytes calldata data
) external {
    if (minter[_msgSender()] == false) revert NoPermission(); // Check
    if (amount == 0) { /* Handle or revert */ } // Consider adding check

    uint256 startTokenId = _idCounter;
    _idCounter += amount; // Effect: Update _idCounter for the entire batch first.
```

```

    for (uint256 i = 0; i < amount; i++) {
        // Interaction: _safeMint itself will have internal effects then potential
interaction.
        _safeMint(to, startTokenId + i, data);
    }
    // _idCounter is already updated.
}

```

Status

Resolved

[L-03] Node1#_setMinter&setBurners&SetUriSetters- No Duplicate Check in similar setter functions

Description

The `_setMinter` internal function (and by extension, the external `setMinters` which calls it in a loop) updates the minter mapping for an account. It does not first check if the new `_permission` being set is identical to the one already stored for that `_account`.

If an admin calls `setMinters` (or `setBurners`, `setUriSetters`), providing an account and a permission status that is already in effect for that account, the state variable will be rewritten with the same value. This redundant `SSTORE` operation consumes unnecessary gas and results in the emission of an unneeded event (e.g., `SetMinter`). The functional impact is negligible, primarily a minor gas inefficiency.

Recommendation

Before assigning the new permission and emitting an event, check if the current permission is different from the new one.

```

// Node1.sol
function _setMinter(address _account, bool _permission) internal {
    if (minter[_account] == _permission) {
        return; // Or revert if explicit feedback is desired for no-op
    }
    minter[_account] = _permission;
}

```

```

    emit SetMinter(_account, _permission);
}
// A similar check should be applied within the loops of setBurners and setUriSetters,
// or their internal helper if they had one like _setBurner.
// For setBurners and setUriSetters, the check would be inside the loop before
// assignment:
// e.g., in setBurners:
// if (burner[_accounts[i]] != _permissions[i]) {
//     burner[_accounts[i]] = _permissions[i];
//     emit SetBurner(_accounts[i], _permissions[i]);
// }

```

Status

Resolved

[L-04] Node1#_setUnlockTransferTime- Missing Input Validation

Description

The `_setUnlockTransferTime` function, accessible via an admin-only external function, allows setting the `unlockTransferTime` for NFTs. It lacks validation for the input `_unlockTransferTime` value, permitting past timestamps or zero.

An administrator could inadvertently set `unlockTransferTime` to a past date (making tokens immediately transferable if they were locked) or to 0. The effect of 0 depends on the `checkLock` modifier logic: `unlockTransferTime > 0 && unlockTransferTime <= block.timestamp`. If `unlockTransferTime` is 0, the first part (`unlockTransferTime > 0`) is false, meaning the else block of `checkLock` (`if (whitelist[from] == false && whitelist[to] == false)` revert `TransferLocked()`;) would always execute, effectively keeping transfers subject to whitelist rules permanently unless the time is updated again. This can lead to misconfigured token transferability.

Recommendation

Add input validation to `_setUnlockTransferTime`. A common practice is to require the time to be in the future, or to clearly define the behavior if 0 is a permissible "special" value (e.g., "never lock based on time" or "defer to whitelist only").

```
// Node1.sol
function _setUnlockTransferTime(uint256 _unlockTransferTime) internal {
    // If 0 is intended to mean "no time-based lock, rely on whitelist"
    if (_unlockTransferTime != 0) {
        require(_unlockTransferTime > block.timestamp, "Node1: Unlock time must be in the
future");
    }
    unlockTransferTime = _unlockTransferTime;
    emit SetUnlockTransferTime(_unlockTransferTime);
}
```

Status

Resolved

Gas

[G-01] Factory#Constructor - Redundant State Assignment of useDAO in constructor

Description

In the `Factory` contract's constructor, the boolean state variable `useDAO` is explicitly assigned the value `false` (`useDAO = false;`). Boolean state variables in Solidity are initialized to `false` by default.

This assignment is redundant and has no functional effect on the contract's logic as `useDAO` would already be `false`. It consumes a negligible amount of extra gas during deployment for the unnecessary `SSTORE` operation.

Recommendation

Remove the redundant line `useDAO = false;` from the constructor in `Factory.sol`.

Status

Resolved

[G-02] Factory - Inefficient Data Location for Read-Only String Parameters

Description

External functions in `Factory.sol` (`addSaleDeployer`, `addTierPriceDeployer`, `addNodeDeployer`) declares their string parameters (e.g., `_description`) with the memory data location. Since these string parameters are only read and not modified within these functions, using `calldata` would be more gas-efficient.

Using memory for read-only external function parameters that are complex types (like strings, arrays, structs) incurs an unnecessary gas cost for copying the data from `calldata` (where arguments from external calls reside) to memory.

Recommendation

Change the data location for read-only string (and other applicable complex type) parameters in external functions from memory to `calldata`.

Status

Resolved

[G-03] Payments#_addTokens - Inefficient Gas Usage Due to Check Order

Description

In the `_addTokens` function (e.g., in `Payments.sol`, used by `Sale1.sol` via `admin` function `addTokens`), the check for `_addedTokens.length() > MAX_PAYMENTS_LENGTH` occurs *after* iterating through the input `_tokens` array and performing state changes (like setting `_tokenAccepted`, `_tokenDecimals`, adding to `_addedTokens` set, emitting events).

If an administrator calls an external function that uses `_addTokens` (e.g., `addTokens` in `Sale1.sol`) with a list of tokens that would cause the total number of accepted payment tokens to exceed `MAX_PAYMENTS_LENGTH`, the transaction will revert. However, gas will have already been consumed for loop iterations and state modifications for each token processed *before* the revert, leading to wasted gas.

Recommendation

Move the `MAX_PAYMENTS_LENGTH` check to the beginning of the `_addTokens` function, before the loop, to fail early.

Status

Resolved

QA

[Q-01] Contracts- Floating Pragma Version

Description

Contracts utilize a floating pragma (e.g., `pragma solidity ^0.8.19;` or `^0.8.13;`). This allows compilation with different patch versions of the specified compiler, potentially leading to the use of a compiler version with undiscovered bugs or slightly different behavior than the one used during development and testing.

Deploying with a compiler version different from the one thoroughly tested can introduce subtle bugs or unexpected behaviors in the smart contracts. This increases deployment risk, although it is typically lower for patch versions.

Recommendation

Lock the pragma to a specific, well-tested Solidity compiler version in all contract files to ensure deterministic and consistent bytecode generation.

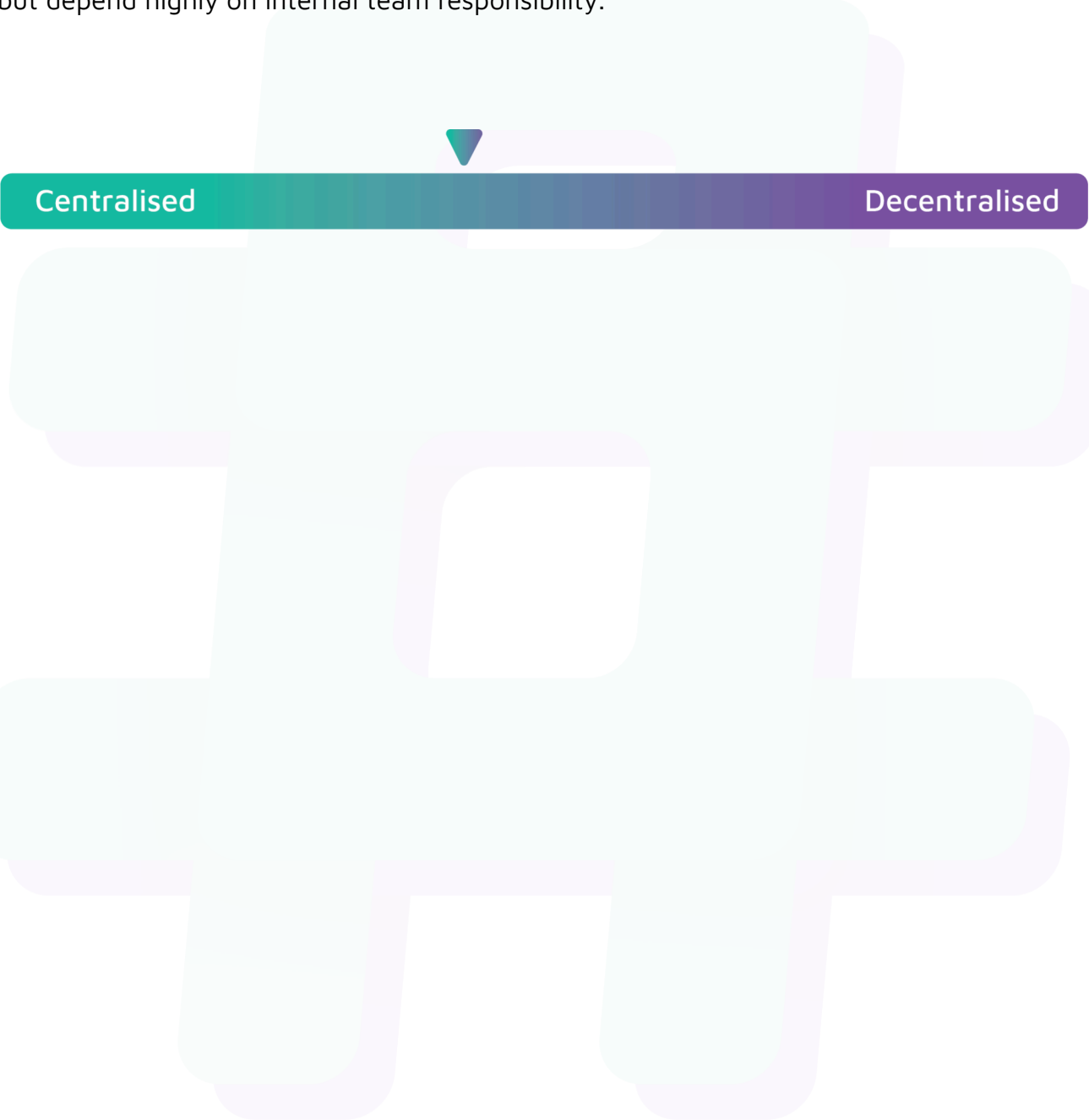
Status

Resolved

Centralisation

The LFG Incorporated project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the LFG Incorporated project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.